

Plano de Voo: Da Intenção à Spec Executável & Cartografia do Domínio: Design Orientado a Agentes — Um Recorte de SDLC AI-first: O Ciclo de Vida do Software na Era dos Agentes

Heverton Eduardo Peres

RESUMO

A especificação executável e o design orientado a fronteiras são a infraestrutura contratual do ciclo de vida de software orientado a agentes. Este recorte investiga como transformar intenção vaga em spec com critérios de aceite mensuráveis e casos de borda explícitos, e como modelar o domínio — vocabulário ubíquo, deep modules e ADRs — para viabilizar execução agêntica paralela sem retrabalho. A análise do dossiê mostra que requisitos vagos tornam-se bloqueadores na era dos agentes, que a rastreabilidade spec-teste sustenta a verificação por evidência e que o investimento em design reduz o custo dominante do ciclo: os tokens gastos em correção. Conclui-se que a disciplina de especificação deixou de ser etapa burocrática para ser o coração do SDLC AI-first.

Palavras-chave: spec executável, critério de aceite, domain-driven design, deep module, ADR.

ABSTRACT

Executable specification and boundary-oriented design are the contractual infrastructure of agent-driven software development. This slice investigates how to turn vague intent into specs with measurable acceptance criteria and explicit edge cases, and how to model the domain — ubiquitous language, deep modules, and ADRs — to enable parallel agentic execution without rework. The dossier analysis shows that vague requirements become blockers in the agent era, that spec-test traceability sustains evidence-based verification, and that design investment reduces the dominant cycle cost: tokens spent on correction. We conclude that specification discipline is no longer a bureaucratic step but the heart of the AI-first SDLC.

Keywords: executable spec, acceptance criteria, domain-driven design, deep module, ADR.

1 INTRODUÇÃO

A transição para o desenvolvimento de software orientado a agentes reposiciona o artefato central do processo. No ciclo clássico, o documento de requisitos era um registro de intenções que dependia da interpretação humana para se tornar software (SOMMERVILLE, 2019). No ciclo AI-first, esse artefato assume uma função nova e mais severa: ele é, ao mesmo tempo, contrato com a máquina que executa, critério de aceite para a verificação e registro do entendimento compartilhado do time (LEHMANN, 2024). A especificação deixa de ser descrição e passa a ser execução potencial.

Esse reposicionamento tem uma consequência imediata: a qualidade da especificação torna-se o fator dominante da qualidade do produto. Requisitos vagos, que no fluxo tradicional eram absorvidos pela inteligência do desenvolvedor, tornam-se bloqueadores quando não há intérprete humano no caminho (JIN, 2024). A literatura recente converge para o conceito de *spec executável* — uma especificação cujos critérios de aceite são verificáveis por máquina e cujos casos de borda são explícitos (MICROSOFT, 2025; HINGEL, 2026).

O problema de pesquisa deste recorte é duplo. Primeiro: como transformar intenção vaga em especificação executável, com requisitos numerados, critérios de aceite mensuráveis e casos de borda explícitos? Segundo: como modelar o domínio — vocabulário ubíquo, fronteiras entre módulos e decisões de arquitetura — de modo que a execução agêntica não produza retrabalho (EVANS, 2003; HODA, 2025).

A hipótese orientadora é que a disciplina de especificação e design, historicamente tratada como etapa preliminar burocrática, torna-se o coração do SDLC AI-first: quanto melhor o contrato, menos ciclos de correção e menos custo de contexto desperdiçado (GRÖPLER, 2024; GURGUL, 2024). O design orientado a fronteiras — módulos profundos, interfaces estáveis e decisões registradas — é a pré-condição estrutural para que agentes paralelos trabalhem sem colidir (MOHAGHEGHI, 2025).

Este artigo está organizado em quatro seções. A metodologia descreve como o recorte foi construído a partir do dossiê da obra-mãe. Os resultados e a discussão sintetizam as evidências sobre *spec executável*, vocabulário ubíquo e registros de decisão de arquitetura (ADR). A conclusão apresenta as implicações práticas para equipes em transição (ANTHROPIC, 2025; ROYCHOUDHURY, 2025).

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

2 METODOLOGIA

Este artigo é um recorte analítico derivado do livro-mãe *SDLC AI-first: O Ciclo de Vida do Software na Era dos Agentes*. Seguindo o protocolo da esteira de produção, o recorte não realizou pesquisa primária: todo o embasamento foi reaproveitado do dossiê da obra-mãe, composto por artigos científicos indexados em arXiv, ICLR e ACM, além de fontes institucionais de provedores (ANTHROPIC, 2025; MICROSOFT, 2025).

O procedimento metodológico seguiu três etapas. Na primeira, identificou-se o recorte temático: os capítulos 3 e 4 da obra-mãe, dedicados respectivamente à transformação de intenção em spec executável e à cartografia do domínio com design orientado a agentes (SOMMERVILLE, 2019; EVANS, 2003). Os pilares previstos no sumário macro — elicitación de intenção, critérios de aceite, deep modules, vocabulário ubíquo e ADRs — orientaram a busca de evidência.

Na segunda etapa, realizou-se consulta ao índice RAG do dossiê da obra-mãe. Para os termos “spec executável”, “critério de aceite”, “domain-driven design”, “deep module” e “ADR”, recuperaram-se os blocos mais relevantes do índice, que fundamentaram a síntese dos resultados (LEHMANN, 2024; HODA, 2025). Essa etapa garante rastreabilidade: nenhuma afirmação deste artigo provém de fonte fora do dossiê.

Na terceira etapa, triangulou-se a literatura acadêmica com a documentação técnica e os relatórios da indústria. O relatório DORA foi usado como fonte sobre o impacto de práticas de qualidade — como testes automatizados e revisão — no throughput e na estabilidade (GOOGLE CLOUD, 2025; FORSGREN, 2018). A documentação de provedores de IA foi empregada para caracterizar o estado da arte da execução agêntica e do protocolo de ferramentas (CLARKE, 2025; ANTHROPIC, 2024).

A análise é qualitativa e interpretativa, com caráter de revisão crítica aplicada. Não foram coletados dados primários nem executados experimentos controlados — coerente com a natureza do recorte, que consolida evidências publicadas (GURGUL, 2024; GRÖPLER, 2024). As limitações dessa escolha são discutidas na seção de resultados.

As citações seguem a norma NBR 10520 (sistema autor-data) e as referências completas estão listadas na seção final, conforme a NBR 6023 (JIMENEZ, 2024; WANG, 2024). O referencial combina fontes revisadas por pares com documentação técnica, permitindo contrastar a visão acadêmica e a visão da indústria (HINGEL, 2026; ROYCHOUDHURY, 2025).

Por fim, registra-se a decisão metodológica de utilizar a metáfora do plano de voo como categoria analítica, herdada do motivo condutor da obra-mãe. A especificação é o plano de voo aprovado antes da decolagem — sem ele, nenhuma fase inicia (MOHAGHEGHI, 2025; YANG, 2024). O uso da metáfora visa facilitar a comunicação dos resultados para profissionais de software em geral.

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

3 RESULTADOS E DISCUSSÃO

A síntese das fontes do dossiê sustenta que a spec executável é o coração do SDLC AI-first. Em primeiro lugar, a literatura converge para a necessidade de critérios de aceite mensuráveis: um requisito só é considerado completo quando sua verificação pode ser automatizada (MICROSOFT, 2025; HINGEL, 2026). Essa exigência elimina, na prática, a classe de requisitos vagos que dominava os documentos tradicionais — “o sistema deve ser rápido” não é um critério, é uma opinião (SOMMERVILLE, 2019; LEHMANN, 2024).

Em segundo lugar, o recorte revela que a elicitação de intenção mudou de natureza. No fluxo clássico, o analista interpretava e traduzia; no fluxo agêntico, a elicitação precisa produzir um artefato que a máquina consiga executar, o que exige técnicas mais estruturadas de extração de requisitos, incluindo questionários, entrevistas adversariais e formalização de casos de borda (JIN, 2024; HODA, 2025). A literatura aponta que quanto mais cedo o caso de borda é explicitado, menor o custo da correção posterior (GRÖPLER, 2024).

Em terceiro lugar, o design orientado a fronteiras emerge como pré-condição da execução paralela. Os resultados do dossiê indicam que módulos profundos — interfaces pequenas escondendo complexidade interna — reduzem o acoplamento e permitem que agentes trabalhem em paralelo sem colidir (EVANS, 2003; MOHAGHEGHI, 2025). O vocabulário ubíquo, por

sua vez, reduz a ambiguidade que os agentes não conseguem negociar: quando o termo “cliente” significa duas coisas, o agente não pergunta, ele adivinha (GURGUL, 2024).

A discussão dos resultados organiza-se em três eixos. O primeiro é a rastreabilidade spec-teste. A literatura recomenda que cada requisito seja ligado a seus casos de teste de forma verificável por máquina, criando uma cadeia contínua entre intenção e verificação (JIMENEZ, 2024; YANG, 2024). Essa cadeia é o que permite à verificação adversarial — tema do próximo recorte da obra — operar sobre evidência, não sobre intenção.

O segundo eixo é o registro de decisões de arquitetura (ADR). Os resultados indicam que ADRs escritos em formato estruturado — contexto, decisão, consequências e alternativas rejeitadas — funcionam como contratos que orientam tanto humanos quanto agentes (ROY-CHOUDHURY, 2025; CLARKE, 2025). Um agente que consulta o ADR sabe por que a decisão foi tomada e evita reabrir debates já encerrados.

O terceiro eixo é a economia de contexto no design. O custo de especificação mal feita não é apenas o retrabalho — é o custo de tokens gastos em ciclos de correção (MOHAGHEGHI, 2025). O dossiê sugere que investir na spec é investir na redução do custo dominante do ciclo: cada hora de elicitação evita dezenas de horas de execução errada (FORSGREN, 2018; HINGEL, 2026).

A síntese permite identificar tensões não resolvidas. A primeira é o risco de especificação excessiva: o esforço de formalizar tudo pode consumir o ganho que a automação deveria trazer (LEHMANN, 2024). A segunda é a dificuldade de manter specs vivas — o documento que se desatualiza vira especificação decorativa, pior que nenhuma (HODA, 2025; GURGUL, 2024). A terceira é a medição do retorno do investimento em design, ainda incipiente na literatura (GRÖPLER, 2024).

Como limitação, registra-se que o recorte se apoia em fontes publicadas entre 2019 e 2026, com predomínio dos anos 2024-2026, o que pode superestimar a velocidade da adoção de spec-driven development. A ausência de dados primários impede afirmações causais (JIN, 2024; MOHAGHEGHI, 2025).

No conjunto, os resultados sustentam que a especificação executável e o design de fronteiras não são etapas burocráticas, mas a infraestrutura do SDLC AI-first: são elas que transformam intenção em contrato verificável (ANTHROPIC, 2025; WANG, 2024). O plano de voo aprovado antes da decolagem é o que distingue uma fase que avança com segurança de uma que decola às cegas (EVANS, 2003; MICROSOFT, 2025).

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

4 CONCLUSÃO

Este recorte confirmou que a spec executável e o design orientado a fronteiras constituem a infraestrutura contratual do SDLC AI-first. A literatura analisada evidencia que critérios de aceite mensuráveis, casos de borda explícitos e vocabulário ubíquo transformam a intenção em contrato verificável por máquina (MICROSOFT, 2025; EVANS, 2003). Sem esse contrato, a execução agêntica produz retrabalho e custo de contexto desperdiçado (JIN, 2024; LEHMANN, 2024).

O estudo também demonstrou que deep modules e ADRs estruturados são as ferramentas que permitem o trabalho paralelo de agentes sem colisão, e que a rastreabilidade spec-teste cria a cadeia de evidência que sustenta a verificação adversarial (MOHAGHEGHI, 2025; ROYCHOUDHURY, 2025). O design deixou de ser etapa preliminar para ser o coração do ciclo (GRÖPLER, 2024).

Para a prática, as recomendações são: formalizar critérios de aceite verificáveis em toda spec, manter o vocabulário ubíquo como glossário vivo e registrar decisões de arquitetura em ADRs estruturados (HODA, 2025; CLARKE, 2025). Para a pesquisa, ficam em aberto o equilíbrio ótimo entre formalização e agilidade, e a medição do retorno do investimento em design (GURGUL, 2024; FORSGREN, 2018). O próximo recorte desta série examina como a verificação adversarial aproveita essa infraestrutura contratual (YANG, 2024; WANG, 2024).

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.